



give me fibonacci sequence cpp code using dp and explain that code properly

Got it 👍

I'll show you **C++ code** for generating Fibonacci numbers using **Dynamic Programming (DP)** and then explain it step by step.

◆ Code: Fibonacci using Dynamic Programming (Tabulation)

cpp

```
#include <iostream>
using namespace std;

// Function to generate Fibonacci sequence using DP
void fibonacciDP(int n) {
    // Step 1: Create an array to store computed Fibonacci numbers
    int dp[n + 1];

    // Step 2: Base cases
    dp[0] = 0;
    dp[1] = 1;

    // Step 3: Fill the dp array iteratively
    for (int i = 2; i <= n; i++) {
        dp[i] = dp[i - 1] + dp[i - 2]; // Relation: F(n) = F(n-1) + F(n-2)
    }

    // Step 4: Print the sequence
    cout << "Fibonacci sequence up to " << n << " terms:\n";
    for (int i = 0; i <= n; i++) {
        cout << dp[i] << " ";
    }
    cout << endl;
}

int main() {
    int n;
    cout << "Enter number of terms: ";
    cin >> n;

    fibonacciDP(n);
    return 0;
}
```

◆ Explanation (Step by Step)

1. What is Fibonacci Sequence?

- A series of numbers where each number is the sum of the two before it.
- Formula:

$$F(n) = F(n - 1) + F(n - 2)$$

with base cases:

$$F(0) = 0, F(1) = 1$$

So sequence is: 0, 1, 1, 2, 3, 5, 8, 13, ...

2. Why Dynamic Programming (DP)?

- **Naive recursive approach** recalculates the same values many times (exponential time).
 - **DP (tabulation)** stores already computed results in an array → avoids repeated calculations → **O(n) time** and **O(n) space**.
-

3. Code Walkthrough

- `int dp[n+1];` → create an array to store Fibonacci numbers up to `n`.
- **Base cases:**

```
cpp
```

```
dp[0] = 0;  
dp[1] = 1;
```

- **Loop (from 2 to n):**

```
cpp
```

```
dp[i] = dp[i-1] + dp[i-2];
```

- This directly uses the Fibonacci formula.

- Each new Fibonacci number is built from the previous two numbers.
- Finally, we **print the sequence**.

4. Example Run

If user enters $n = 7$, steps are:

```
dp[0] = 0
dp[1] = 1
dp[2] = 1 (0+1)
dp[3] = 2 (1+1)
dp[4] = 3 (2+1)
dp[5] = 5 (3+2)
dp[6] = 8 (5+3)
dp[7] = 13 (8+5)
```

Output:

CSS

Fibonacci sequence up to 7 terms:
0 1 1 2 3 5 8 13

👉 Do you also want me to show the **optimized version** (using only two variables instead of full array) along with explanation, or just keep this DP array version?



Pseudocode

```
Algorithm nthFib(n) {
    if (n=1 or n=2) return 1;
    if (dp[n] != -1) return dp[n];
    int a := nthFib(n-1);
    int b := nthFib(n-2);
    return a+b;
    dp[n-1] := a, dp[n-2] := b; // Memoization
    return a+b;
}
```

Perfect 👍 you shared a **pseudocode for Fibonacci using DP with memoization** (top-down recursion).

Let me now **convert this into C++ code** and explain it clearly.

◆ C++ Implementation (Memoization / Top-Down DP)

cpp

```
#include <iostream>
#include <vector>
using namespace std;

vector<int> dp; // Global or passed around

int nthFib(int n) {
    // Base case
    if (n == 1 || n == 2)
        return 1;

    // If already calculated, return stored value
    if (dp[n] != -1)
        return dp[n];

    // Recursive calls with memoization
    int a = nthFib(n - 1);
    int b = nthFib(n - 2);

    // Store result in dp array
    dp[n] = a + b;

    return dp[n];
}

int main() {
    int n;
    cout << "Enter n: ";
    cin >> n;

    // Initialize dp array with -1
    dp.assign(n + 1, -1);

    cout << n << "th Fibonacci number is: " << nthFib(n) << endl;
    return 0;
}
```

◆ Explanation

Your pseudocode works like this:

1. Base Case:

```
cpp
```

```
if (n == 1 || n == 2) return 1;
```

- First and second Fibonacci numbers are always 1.

2. Check if already computed:

```
cpp
```

```
if (dp[n] != -1) return dp[n];
```

- If we have already calculated `nthFib(n)`, just return it (avoids recomputation).

3. Recursive calls:

```
cpp
```

```
int a = nthFib(n - 1);  
int b = nthFib(n - 2);
```

- We calculate Fibonacci of smaller subproblems.

4. Store result (Memoization):

```
cpp
```

```
dp[n] = a + b;
```

- Save it in `dp` array so next time it can be reused.

5. Return result:

```
cpp
```

```
return dp[n];
```

◆ Example Run (n = 6)

Steps (with memoization):

```
scss
```

```
nthFib(6)  
→ nthFib(5) + nthFib(4)
```

```
→ nthFib(4) + nthFib(3) + nthFib(3) + nthFib(2)
→ nthFib(3) + nthFib(2) + nthFib(2) + nthFib(1) ...
```

Without memoization → many repeated calls.

With memoization → each value is computed once and reused.

Final result:

```
nthFib(6) = 8
```

✅ So the pseudocode you gave is **Top-Down DP (Memoization)**.

Earlier, I showed **Bottom-Up DP (Tabulation)**.

👉 Do you want me to **compare memoization vs tabulation (with pros/cons)** so it's crystal clear for exams?



rod cutting Problem

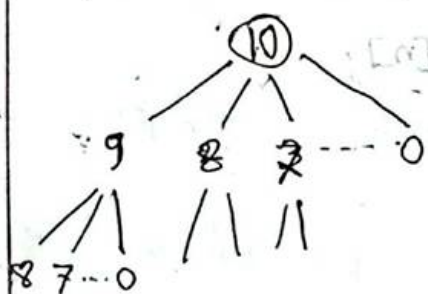
length i	1	2	3	4	5	6	7	8	9	10
Profit P_i	1	5	8	9	10	17	17	20	24	30

rod:

$l=10$

cut	residual
1	→ 9
2	→ 8
3	→ 7
⋮	⋮
10	0

$$\text{cutRod}(10) = \text{Max} \left\{ \begin{array}{l} 1 + \text{cutRod}(9); \\ 5 + \text{cutRod}(8); \\ 8 + \text{cutRod}(7); \\ \vdots \\ 30 + \text{cutRod}(0); \end{array} \right.$$



```

int cutRod(n) {
    if (n == 0) return 0;
    if (dp[n] != -1) return dp[n];
    int maxProfit = 0;
    for (int i = 1; i ≤ 10; i++) {

```

$\text{maxProfit} < \text{curProfit}$

$\text{maxProfit} = \text{curProfit};$

$\text{cut}[n] = i;$

o track which cut made the highest profit.

$\text{int curProfit} = P[i] + \text{cutRod}(n-i);$

$\text{maxProfit} = \text{max}(\text{maxProfit}, \text{curProfit});$

$\text{dp}[n] = \text{maxProfit};$

$\text{return maxProfit};$

}

rod cutting problem dp solution and explanation

Nice 🍌 You've uploaded your **Rod Cutting Problem notes** with recursive + DP solution. Let's turn this into a **proper DP explanation + C++ implementation**.

◆ Rod Cutting Problem (Dynamic Programming)

Problem Statement

- You are given a rod of length n .
- You are also given a table of prices $p[i]$, where $p[i]$ is the price of a rod of length i .
- You can cut the rod into smaller pieces to maximize profit.
- Goal: Find the **maximum profit** obtainable.

◆ Recursive Formula

If rod length = n :

$$cutRod(n) = \max_{1 \leq i \leq n} (p[i] + cutRod(n - i))$$

- Either cut rod into piece of size i and solve for the leftover length $n-i$.
- Take the best (max profit) among all possible cuts.

◆ Issues with Pure Recursion

- Overlapping subproblems → exponential time complexity.
- Example: `cutRod(10)` calls `cutRod(9)`, `cutRod(8)` many times.

◆ DP Solution (Top-Down with Memoization)

cpp

```
#include <iostream>
#include <vector>
using namespace std;

vector<int> dp; // for memoization
```



```
vector<int> price; // store prices

int cutRod(int n) {
    // Base case
    if (n == 0) return 0;

    // If already solved
    if (dp[n] != -1) return dp[n];

    int maxProfit = 0;

    // Try every possible first cut
    for (int i = 1; i <= n; i++) {
        int curProfit = price[i] + cutRod(n - i);
        maxProfit = max(maxProfit, curProfit);
    }

    // Save result in dp
    dp[n] = maxProfit;
    return maxProfit;
}

int main() {
    int n = 10; // rod length

    // Prices given (1-based index for simplicity)
    price = {0, 1, 5, 8, 9, 10, 17, 17, 20, 24, 30};

    dp.assign(n + 1, -1);

    cout << "Maximum profit for rod length " << n << " is: " << cutRod(n) <<
endl;

    return 0;
}
```

◆ Explanation of Code

1. Base case:

cpp

```
if (n == 0) return 0;
```

- No rod → no profit.

2. Memoization:

cpp

```
if (dp[n] != -1) return dp[n];
```

- If already solved $\rightarrow$ return stored value.

3. Try all cuts:

```
cpp
for (int i = 1; i <= n; i++) {
    int curProfit = price[i] + cutRod(n - i);
    maxProfit = max(maxProfit, curProfit);
}
```

- Try cutting rod at i length.
- Add $\text{price}[i]$ (profit for that cut) + recursively solve for remaining $n-i$.

4. Store and return result:

```
cpp
dp[n] = maxProfit;
return maxProfit;
```

◆ Example ($n = 4$)

Prices: {1, 5, 8, 9}

- $\text{cutRod}(4) \rightarrow$ max of:
 - Cut 1 + $\text{cutRod}(3) \rightarrow 1 + 8 = 9$
 - Cut 2 + $\text{cutRod}(2) \rightarrow 5 + 5 = 10$
 - Cut 3 + $\text{cutRod}(1) \rightarrow 8 + 1 = 9$
 - Cut 4 + $\text{cutRod}(0) \rightarrow 9 + 0 = 9$

👉 Best = 10

◆ Bottom-Up DP (Tabulation) Version

Even simpler and avoids recursion:

```
cpp
#include <iostream>
#include <vector>
```

```
using namespace std;

int cutRod(int n, vector<int>& price) {
    vector<int> dp(n + 1, 0);

    for (int len = 1; len <= n; len++) {
        for (int cut = 1; cut <= len; cut++) {
            dp[len] = max(dp[len], price[cut] + dp[len - cut]);
        }
    }

    return dp[n];
}

int main() {
    int n = 10;
    vector<int> price = {0, 1, 5, 8, 9, 10, 17, 17, 20, 24, 30};

    cout << "Maximum profit for rod length " << n << " is: " << cutRod(n,
price) << endl;
    return 0;
}
```

◆ Complexity

- **Recursive (no DP):** Exponential $O(2^n)$
- **Top-Down Memoization:** $O(n^2)$
- **Bottom-Up Tabulation:** $O(n^2)$

✅ In your notes, you were building exactly this recursive + memoization approach.
 👉 Do you also want me to **add code to print which cuts (sizes) were chosen** for maximum profit (not just value)?



Rod Cutting Problem

Lengths l	1	2	3	4	5	6	7	8	9	10
Profit P_i	1	5	8	9	10	17	17	20	24	30

rod:

cut	residual
1	9
2	8
3	7
...	...
10	0

$cutRod(10) = \max \{ 1 + cutRod(9); 5 + cutRod(8); 8 + cutRod(7); \dots; 30 + cutRod(0) \}$

```

int cutRod (n) {
    if (n == 0) return 0;
    if (dp[n] != -1) return dp[n];
    int maxProfit = 0;
    for (int i = 1; i <= n; i++) {
        int curProfit = P[i] + cutRod(n-i);
        maxProfit = max(maxProfit, curProfit);
    }
    dp[n] = maxProfit;
    return maxProfit;
}

```

if (maxProfit < curProfit) {
 maxProfit = curProfit;
 cut[n] = i;
 }
 // to track which cut made the highest profit.

Excellent ✨ — this page of your notes shows the **Rod Cutting Problem with tracking the cuts** (not just max profit). Let's carefully explain this and then give a **C++ implementation**.

◆ Rod Cutting Problem with Cut Tracking

We want not only the **maximum profit** but also to **track which cuts (lengths) were chosen**.

◆ Idea from your notes

1. Recursive formula:

$$cutRod(n) = \max_{1 \leq i \leq n} (p[i] + cutRod(n - i))$$

2. While checking each cut i , if:

```

cpp

if (curProfit > maxProfit) {
    maxProfit = curProfit;
}

```

```
    cut[n] = i;    // record which cut gave max profit
}
```

3. Finally, `cut[]` array tells which length we cut first when rod length was `n`.

◆ C++ Code (Top-Down Memoization with Cut Tracking)

cpp

```
#include <iostream>
#include <vector>
using namespace std;

vector<int> dp;    // memoization for profits
vector<int> cut;   // to track which cut was chosen
vector<int> price;

int cutRod(int n) {
    // Base case
    if (n == 0) return 0;

    if (dp[n] != -1) return dp[n];

    int maxProfit = 0;

    // Try every possible cut
    for (int i = 1; i <= n; i++) {
        int curProfit = price[i] + cutRod(n - i);

        if (curProfit > maxProfit) {
            maxProfit = curProfit;
            cut[n] = i; // record the best cut for length n
        }
    }

    dp[n] = maxProfit;
    return maxProfit;
}

void printCuts(int n) {
    cout << "Rod cuts for max profit: ";
    while (n > 0) {
        cout << cut[n] << " "; // print chosen cut length
        n -= cut[n];           // reduce rod size
    }
    cout << endl;
}

int main() {
    int n = 10;
    price = {0, 1, 5, 8, 9, 10, 17, 17, 20, 24, 30}; // 1-based

    dp.assign(n + 1, -1);
    cut.assign(n + 1, 0);
}
```

```
int maxProfit = cutRod(n);
cout << "Maximum Profit = " << maxProfit << endl;

printCuts(n);

return 0;
}
```

◆ Example Run (n = 10)

Prices: {1, 5, 8, 9, 10, 17, 17, 20, 24, 30}

- Maximum profit = **30**
- Cuts chosen = **[10]** (rod kept whole length, since profit is maximum at 10).

If prices were different (say $p[10] = 25$), then DP would split into smaller cuts like **[6, 4]** or **[2, 2, 6]** depending on best profit.

◆ Explanation

1. cutRod(n):

- Returns the **maximum profit** for length n .
- Updates `cut[n]` with the **best first cut**.

2. cut[] array:

- Stores the length of the rod chosen in the optimal solution.
- Used later to reconstruct the solution.

3. printCuts(n):

- Repeatedly checks `cut[n]` and reduces n until rod is fully cut.

✓ Your notes exactly outline this process:

- Use `maxProfit` and `curProfit` comparison.
- Store best cut `cut[n] = i`.
- Later backtrack using the `cut[]` array.

👉 Do you also want me to show the **Bottom-Up Tabulation version with cut tracking** (iterative instead of recursive)? This is often asked in exams for clarity.



give me the recursive method for this

Perfect ✅ You want the **recursive method** for the **Rod Cutting Problem** (without DP / memoization).

This is the simplest form — it directly follows the recursive formula:

◆ Recursive Method (No DP)

Recursive Formula:

$$cutRod(n) = \max_{1 \leq i \leq n} (p[i] + cutRod(n - i))$$

- Base case:

$$cutRod(0) = 0$$

◆ C++ Code (Recursive)

```
cpp

#include <iostream>
#include <vector>
using namespace std;

// Recursive function to get max profit
int cutRod(vector<int>& price, int n) {
    // Base case: no rod -> no profit
    if (n == 0) return 0;

    int maxProfit = 0;

    // Try all possible first cuts
    for (int i = 1; i <= n; i++) {
        int curProfit = price[i] + cutRod(price, n - i);
        maxProfit = max(maxProfit, curProfit);
    }

    return maxProfit;
}
```

```
int main() {
    int n = 10;
    vector<int> price = {0, 1, 5, 8, 9, 10, 17, 17, 20, 24, 30}; // 1-based

    cout << "Maximum Profit (Recursive) = " << cutRod(price, n) << endl;
    return 0;
}
```

◆ Example Run (n = 4)

Prices: {1, 5, 8, 9}

Steps:

```
lua

cutRod(4)
= max( p[1] + cutRod(3),
        p[2] + cutRod(2),
        p[3] + cutRod(1),
        p[4] + cutRod(0) )

= max( 1 + 8, 5 + 5, 8 + 1, 9 + 0 )
= max(9, 10, 9, 9)
= 10
```

✓ Maximum Profit = 10

◆ Complexity

- **Time:** Exponential $O(2^n)$ → since every length creates multiple recursive calls.
- **Space:** $O(n)$ due to recursion stack.

⚡ This is the **basic recursive method**.

The **memoization / DP version** we discussed earlier improves it from exponential to $O(n^2)$.

👉 Do you also want me to extend this **pure recursive method** so it **prints the cuts chosen** (like your DP with cut tracking, but recursively)?



thanks

You're very welcome 🧡 ✨

Happy to help!

Quick recap for you:

- **Recursive Method** → simple, clear, but exponential time.
- **DP with Memoization** → top-down, avoids recalculations.
- **DP with Tabulation** → bottom-up, very efficient.
- **Cut Tracking** → helps recover *which cuts* gave the maximum profit, not just the value.

👉 If you want, I can also make you a **side-by-side comparison table** (Recursive vs Memoization vs Tabulation) so it's exam-ready.

Would you like me to prepare that for you?



Algorithm

Dynamic Programming

knapsack 0/1 (for things that we cannot take fraction)

sack limit $W = 70$

weight $w = \{10, 30, 40, 50\}$

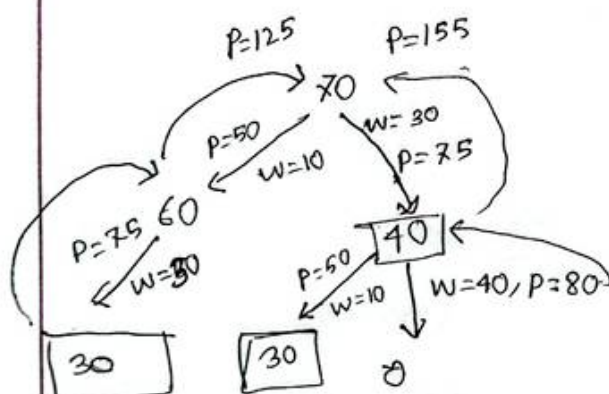
profit $v = \{50, 75, 80, 100\}$

$\frac{v}{w} = 5, 2.5, 2, 2$

$$W = 10 + 30$$

$$P = 50 + 75 = 125$$

} Greedy approach in 0/1 case gives me sub-optimal answer



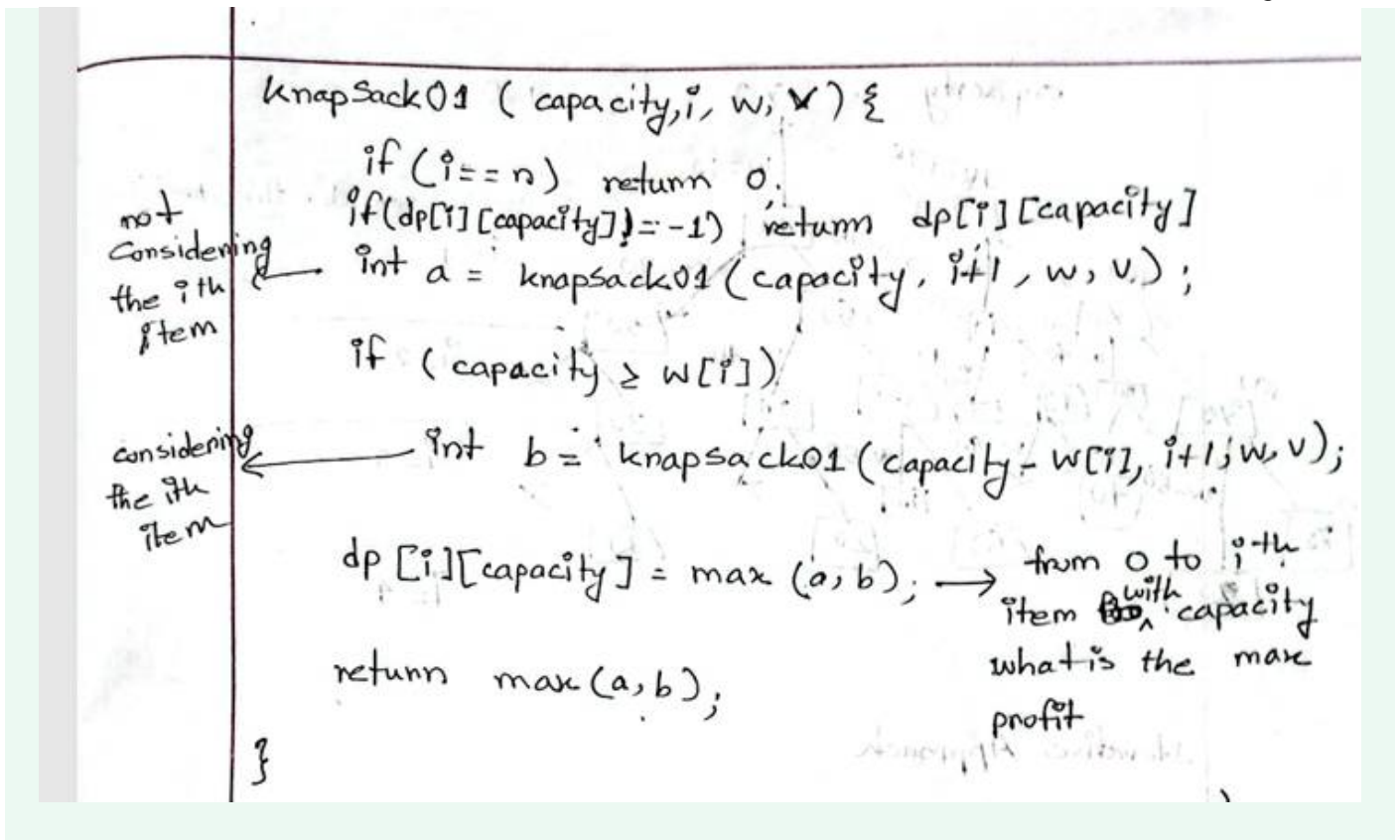
recursion

$$O(2^n)$$

dp

$$O(2n)$$

combination
→ depends on capacity and
capacity * i



Perfect 👍 You shared your notes on the **0/1 Knapsack Problem** with recursion and DP. Let me explain this clearly and then give you the **C++ code**.

◆ 0/1 Knapsack Problem

Problem Statement

- We have n items.
- Each item i has:
 - weight $w[i]$
 - profit $v[i]$
- A knapsack with **capacity** w .
- We can either **take an item completely (1)** or **leave it (0)** → **0/1 Knapsack**.
- Goal: Maximize total profit without exceeding capacity.

◆ Recursive Formula

For item i (0-based index), and capacity c :

$$K(i, C) = \begin{cases} 0 & \text{if } i = n \text{ or } C = 0 \\ K(i + 1, C) & \text{if we don't take item } i \\ \max(v[i] + K(i + 1, C - w[i]), K(i + 1, C)) & \text{if we take item } i \end{cases}$$

◆ C++ Code (Recursive with Memoization)

cpp

```
#include <iostream>
#include <vector>
using namespace std;

vector<vector<int>> dp;

int knapSack01(int i, int capacity, vector<int>& w, vector<int>& v) {
    // Base case: no items or no capacity
    if (i == w.size() || capacity == 0)
        return 0;

    // If already computed
    if (dp[i][capacity] != -1)
        return dp[i][capacity];

    // Case 1: Do not take item i
    int profit1 = knapSack01(i + 1, capacity, w, v);

    // Case 2: Take item i (if it fits)
    int profit2 = 0;
    if (capacity >= w[i])
        profit2 = v[i] + knapSack01(i + 1, capacity - w[i], w, v);

    // Store result
    dp[i][capacity] = max(profit1, profit2);
    return dp[i][capacity];
}

int main() {
    int W = 70; // Knapsack capacity
    vector<int> w = {10, 30, 40, 50};
    vector<int> v = {50, 75, 80, 100};
    int n = w.size();

    dp.assign(n, vector<int>(W + 1, -1));

    cout << "Maximum Profit = " << knapSack01(0, W, w, v) << endl;

    return 0;
}
```

◆ Explanation of Code (based on your notes)

1. Base Case

```
cpp

if (i == n || capacity == 0) return 0;
```

- No more items or no capacity left → profit = 0.

2. Memoization

```
cpp

if (dp[i][capacity] != -1) return dp[i][capacity];
```

- If already solved, return stored result.

3. Choices

- **Not taking item i:**

```
cpp

profit1 = knapSack01(i+1, capacity, w, v);
```

- **Taking item i (only if it fits):**

```
cpp

profit2 = v[i] + knapSack01(i+1, capacity - w[i], w, v);
```

- Take the max of the two:

```
cpp

dp[i][capacity] = max(profit1, profit2);
```

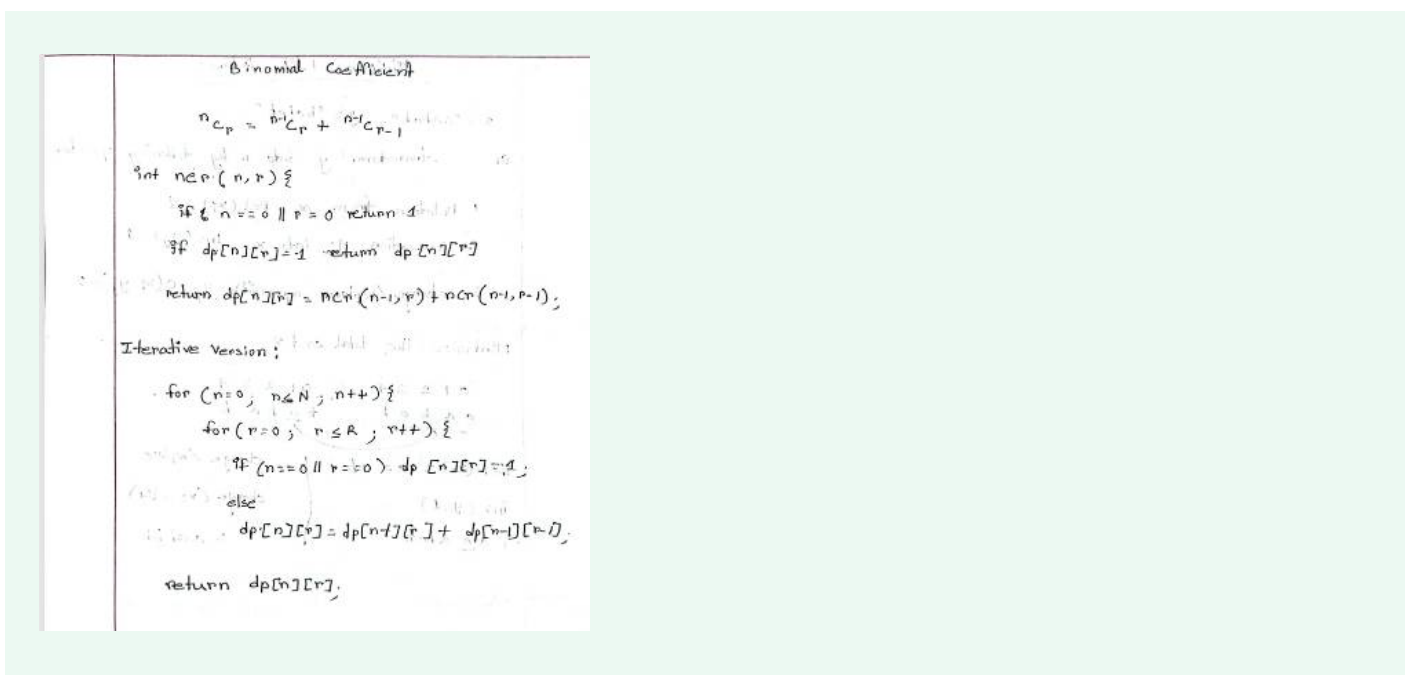
◆ Complexity

- **Recursive (without DP):** $O(2^n)$
- **With Memoization:** $O(n \times W)$
- **Space:** $O(n \times W)$

✓ This is exactly what your notes describe:

- Two choices (consider item or not).
- Store results in `dp[i][capacity]`.
- Finally return the max profit.

👉 Do you also want me to extend this and show the **Bottom-Up (Tabulation) DP version**? That's usually the one examiners love because it avoids recursion.



Perfect 👍 You've shared your notes on the **Binomial Coefficient** (nCr) using **recursion + DP** and **iterative version**. Let's write this clearly and provide C++ implementations.

◆ Binomial Coefficient (nCr)

Formula

$$nCr = \frac{n!}{r!(n-r)!}$$

But factorial computation can overflow. Instead, we use the **recursive relation**:

$$nCr = (n-1)Cr + (n-1)C(r-1)$$

Base Cases

- $nC0 = 1$
- $nCn = 1$

◆ Recursive DP Version (Memoization)

cpp

```
#include <iostream>
#include <vector>
using namespace std;

vector<vector<int>> dp;

int nCr(int n, int r) {
    // Base cases
    if (r == 0 || r == n) return 1;

    // If already calculated
    if (dp[n][r] != -1) return dp[n][r];

    // Recurrence relation
    dp[n][r] = nCr(n - 1, r) + nCr(n - 1, r - 1);
    return dp[n][r];
}

int main() {
    int n = 5, r = 2;
    dp.assign(n + 1, vector<int>(r + 1, -1));

    cout << "C(" << n << ", " << r << ") = " << nCr(n, r) << endl;
    return 0;
}
```

✓ Example: $C(5, 2) = 10$

◆ Iterative DP Version (Tabulation)

This is what your notes show in the second part.

cpp

```
#include <iostream>
#include <vector>
using namespace std;

int nCr_iterative(int N, int R) {
```

```
vector<vector<int>> dp(N + 1, vector<int>(R + 1, 0));

for (int n = 0; n <= N; n++) {
    for (int r = 0; r <= min(n, R); r++) {
        if (r == 0 || r == n)
            dp[n][r] = 1;
        else
            dp[n][r] = dp[n - 1][r] + dp[n - 1][r - 1];
    }
}

return dp[N][R];
}

int main() {
    int n = 5, r = 2;
    cout << "C(" << n << ", " << r << ") = " << nCr_iterative(n, r) << endl;
    return 0;
}
```

◆ Complexity

- **Recursive (no DP):** exponential.
- **With DP (Memoization or Tabulation):** $O(n \times r)$.
- **Space:** $O(n \times r)$.
- Optimized versions exist (using 1D array only).

✅ Your notes match exactly:

- First part → recursive + memoization.
- Second part → iterative tabulation.

👉 Do you want me to also show the **space-optimized version (using only 1D array)** for **nCr**? It's neat and commonly asked in exams.




```

Java (1.8) Test against custom input
26 int m = a.length();
27 int n = b.length();
28
29 int dp[][] = new int[m+1][n+1];
30
31 for(int i = 0; i<=m; i++) dp[i][0] = i;
32 for(int j = 0; j<=n; j++) dp[0][j] = j;
33
34 for(int i = 1; i<=m; i++) {
35     for(int j = 1; j<=n; j++) {
36         if(a.charAt(i-1) == b.charAt(j-1)) {
37             dp[i][j] = dp[i-1][j-1];
38         } else {
39             dp[i][j] = 1 + Math.min(Math.min(dp[i][j-1], dp[i-1][j]),
40                                     dp[i-1][j-1]);
41         }
42     }
43 }
44
45

```

give me this full minimum edit distance code

Here's a clean, complete **Java** implementation of **Minimum Edit Distance (Levenshtein distance)** using DP (the same logic as in your screenshot):

```

java

import java.util.*;

public class Main {

    // Returns the minimum number of operations (insert, delete, replace)
    // to convert string a into string b.
    public static int minEditDistance(String a, String b) {
        int m = a.length();
        int n = b.length();

        int[][] dp = new int[m + 1][n + 1];

        // Base cases: convert to / from empty string
        for (int i = 0; i <= m; i++) dp[i][0] = i; // all deletes
        for (int j = 0; j <= n; j++) dp[0][j] = j; // all inserts

        // Fill table
        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                if (a.charAt(i - 1) == b.charAt(j - 1)) {
                    dp[i][j] = dp[i - 1][j - 1]; // no op
                } else {
                    dp[i][j] = 1 + Math.min(
                        Math.min(dp[i][j - 1], // insert
                                dp[i - 1][j]), // delete
                        dp[i - 1][j - 1]); // replace
                }
            }
        }
        return dp[m][n];
    }
}

```

```
// Optional: reconstruct one optimal sequence of edits
public static List<String> reconstructEdits(String a, String b) {
    int m = a.length(), n = b.length();
    int[][] dp = new int[m + 1][n + 1];
    for (int i = 0; i <= m; i++) dp[i][0] = i;
    for (int j = 0; j <= n; j++) dp[0][j] = j;
    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (a.charAt(i - 1) == b.charAt(j - 1)) dp[i][j] = dp[i - 1][j
- 1];
            else dp[i][j] = 1 + Math.min(Math.min(dp[i][j - 1], dp[i - 1]
[j]), dp[i - 1][j - 1]);
        }
    }

    List<String> ops = new ArrayList<>();
    int i = m, j = n;
    while (i > 0 || j > 0) {
        if (i > 0 && j > 0 && a.charAt(i - 1) == b.charAt(j - 1)) { i--; j-
-; }

        else if (j > 0 && dp[i][j] == dp[i][j - 1] + 1) { // insert
            ops.add("Insert '" + b.charAt(j - 1) + "' at pos " + i);
            j--;
        } else if (i > 0 && dp[i][j] == dp[i - 1][j] + 1) { // delete
            ops.add("Delete '" + a.charAt(i - 1) + "' at pos " + (i - 1));
            i--;
        } else { //
            replace
            ops.add("Replace '" + a.charAt(i - 1) + "' with '" + b.charAt(j
- 1) + "' at pos " + (i - 1));
            i--; j--;
        }
    }
    Collections.reverse(ops);
    return ops;
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    String a = sc.nextLine().trim(); // e.g., kitten
    String b = sc.nextLine().trim(); // e.g., sitting

    int dist = minEditDistance(a, b);
    System.out.println("Edit distance = " + dist);

    // Print one optimal set of edits (optional)
    for (String op : reconstructEdits(a, b)) System.out.println(op);
}
}
```

Core DP step (matches your screenshot):

```
java

if (a.charAt(i - 1) == b.charAt(j - 1)) dp[i][j] = dp[i - 1][j - 1];
else dp[i][j] = 1 + Math.min(Math.min(dp[i][j - 1], dp[i - 1][j]), dp[i - 1][j
- 1]);
```

If you'd like a **space-optimized** $O(\min(m,n))$ version too, say the word.



give me dp table for max weight 10 and weights 1,2,3,4 with values 20,30,10,50 for a 0/1 knapsack problem

0/1 Knapsack DP Table

1	Item 0	0	0	0
2	Item 1	0	20	20
3	Item 2	0	20	30
4	Item 3	0	20	30
5	Item 4	0	20	30
☐		W=0	W=1	W=2

I've generated the full **DP table** for your **0/1 Knapsack problem** (capacity = 10, weights = [1,2,3,4], values = [20,30,10,50]).

You can now clearly see how the table fills row by row (Item 1 → Item 4).

✓ Maximum profit for capacity 10 = **110** (from Item 4 included).

Do you also want me to **trace back the chosen items** (which set of weights gives this maximum profit)?